



SENIOR PRODUCT DESIGNER · HOME ASSIGNMENT

Designing the Integrations page for Act Security

The page where a security engineer connects their tools – and sees exactly what access they grant before anything happens.

Live React prototype

Trust-first flow

Setup Copilot

Live prototype act-senior-prototype.lovable.app

Design system reutkabilo-spec.github.io/Act-Security

THE REFRAME THAT DRIVES THE PAGE

Integrations aren't just a marketplace. They're Act's activation surface.

Legacy tools treat integrations as data sources feeding a dashboard. Act's preemptive bet reframes the page as an active setup surface – not a wall of logos.

YESTERDAY · REACT

- Detect → triage → respond, after exposure
- Integrations = sources feeding a dashboard
- Success = more findings, more visibility
- A passive logo marketplace

ACT · PREEMPT

- Map access → harden → close risky paths
- Integrations = how Act receives context and enables action
- Success = least privilege by default, paths closed
- An active activation surface

Based on Act's public positioning – treated as my product hypothesis, not inside knowledge.

WHO I DESIGNED FOR

The operator who activates value – not a casual browser

The page is for the person building the setup and putting their name on it – technical, precise, trustworthy, never dumbed down.

PRIMARY

Cloud Security Engineer

Technical operator. Owns credentials, scopes, connection health. Designs the setup.

SECONDARY

CISO

Coverage, confidence, risk reduction. Reads status, not config.

SECONDARY

DevOps / Platform

Cloud creds, IaC pipelines, environment scoping.

CONSUMER

SOC analyst

Consumes signal downstream – does not configure integrations.

WHO I DESIGNED FOR - AND WHAT THEY ACTUALLY FEEL

Setup is a high-trust, high-risk moment

The operator is a Cloud Security Engineer – technical, precise, accountable – granting a security platform access to highly sensitive cloud and identity systems. The dominant emotion isn't curiosity; it's permission anxiety.

Permission anxiety

“What exactly can it touch? Can it change things? Am I widening my attack surface by connecting a security tool?”

Setup anxiety

“Did I scope this correctly? What breaks if I get a credential or a write scope wrong?”

Recovery anxiety

“If this fails, or I revoke it later, what stops working – and will I know in time?”

Design principle Answer the trust questions before asking for a single credential.

WHAT I LEARNED FROM BENCHMARKS

Patterns I borrowed – and where they show up

I studied how mature platforms handle integrations, then translated the recurring patterns into Act-specific decisions.

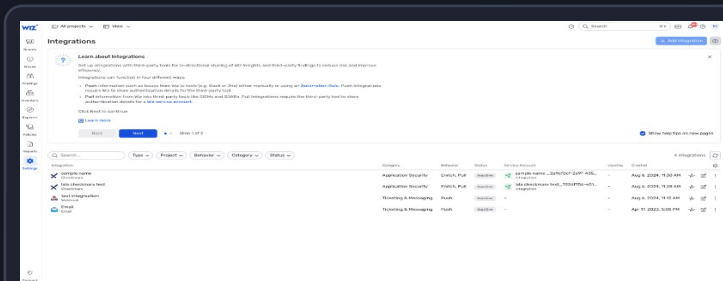
PRODUCT PATTERN	WHAT I BORROWED FOR ACT
Wiz · Orca	Frame integrations by security value – not a wall of vendor logos.
Microsoft Sentinel	Connector health matters after setup, not only during it.
Datadog	Technical setup needs strong guidance, credential clarity and recovery.
Okta · Slack	Consent should be simple, explicit and reversible.
ServiceNow	Ownership, audit trail and admin visibility build trust.

Patterns observed from public product behaviour and docs – adapted, not copied.

VISUAL RESEARCH · THE SCREENS I STUDIED

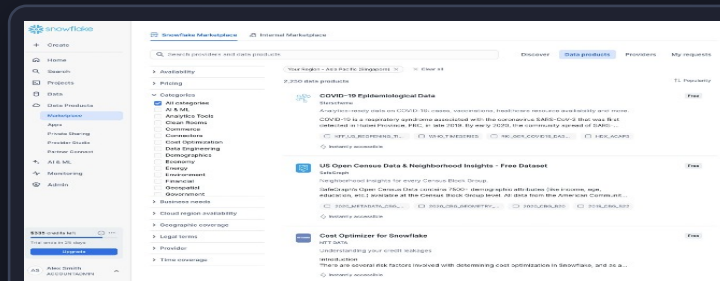
I pulled the real screens apart before drawing mine

Each gave one concrete pattern – and together they shaped one decision: make setup guided, reversible, and status-driven.



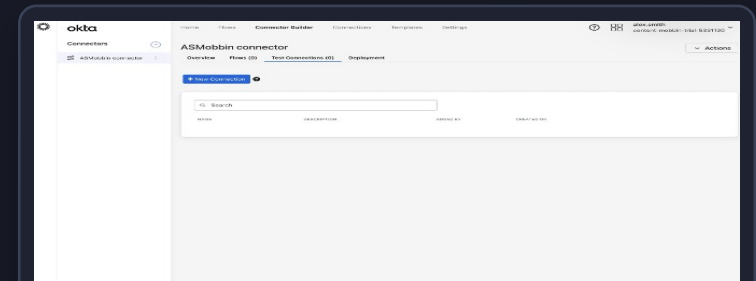
Wiz

Catalog as ecosystem coverage – not a logo wall.



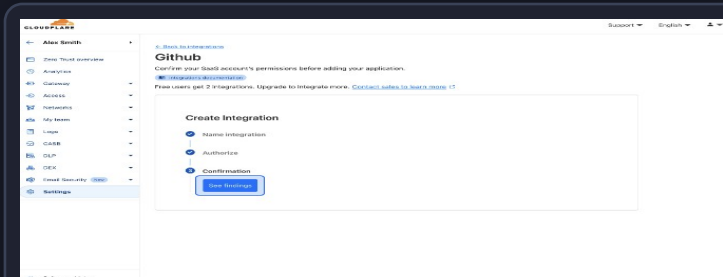
Snowflake

Faceted marketplace: search + categories at scale.



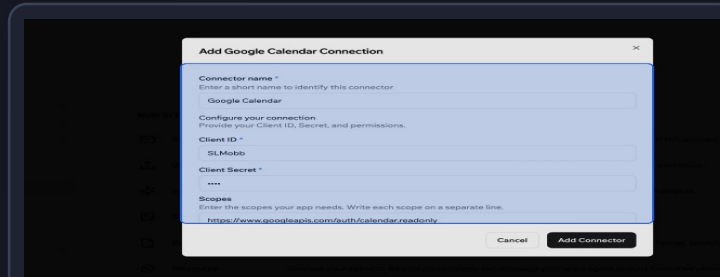
Okta

Lifecycle tabs: Overview · Flows · Test · Deploy.



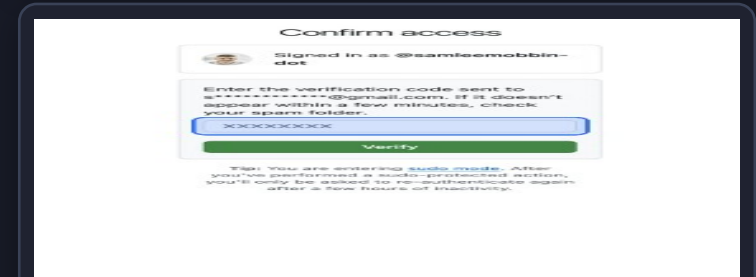
Cloudflare

Vertical stepper: Name → Authorize → Confirm.



Connector OAuth

Scopes shown in the consent modal before granting.



GitHub

Sudo-mode re-confirm before a sensitive action.

From trust needs to concrete design responses

Each need maps to a specific affordance you can see in the live prototype.

USER NEED	WHY IT MATTERS	MY DESIGN RESPONSE
Know what is accessed before consenting	Granting cloud access can feel risky and hard to reverse	“Data Act will access” + “Act will never...” shown pre-credentials
Least privilege by default	Write access widens blast radius	Read-only recommended; write scopes optional, off by default
Confidence it is wired correctly	Silent misconfig = security blind spot	Test connection before enabling; “Connected” = data flowing
Safe recovery	Fear of breaking production	Disconnect names what stops; revoke anytime; reconnect keeps config
Accountability	Enterprise & audit requirements	Owner, required role, last change, audit log per integration

Five principles I designed every screen against

Each principle answers an operator anxiety – and shows up, made concrete, in the prototype.

01

Progressive disclosure

One decision at a time. Day-0 shows a single next step; depth unfolds only when asked.

02

Trust before credentials

What Act reads, what it never does, and least-privilege defaults – shown before any secret.

03

Honest status

“Connected” means data is flowing – not “saved.” Every failure names its cause and fix.

04

Recoverable by design

Reconnect keeps config; disconnect names what stops; revoke anytime. No dead ends.

05

AI guides, never autopilots

Setup Copilot recommends, explains and unblocks. The operator approves every access decision.

Designed for faster time-to-first-value · fewer failed setups · connector health stays visible · less support load

Grouped by what Act does – not A to Z

The catalog is organized into three dependency tiers, built bottom-up. Foundation is the only mandatory tier – connect it and Act is set up. Context and Action enrich from there, so progress never demands all 17 connectors.

FOUNDATION

Core visibility Cloud & identity

AWS · Okta · Microsoft Entra ID · Azure · GCP

MANDATORY

100% when all connected

↓ feeds risk context

CONTEXT

Risk context SIEM & evidence

Splunk · Microsoft Sentinel · QRadar · Elastic Security

ENRICHES

optional

↓ routes to action

ACTION

Response workflow Ticketing & remediation

Jira · ServiceNow · Slack · Teams · XSOAR · Tines · GitHub

AUTOMATES

optional

One clear next step, depth on demand

Start with one move

A new account shows a single recommended step – connect AWS – not everything at once.

Show the path

It maps the route that step kicks off: Foundation, then Context, then Action.

Detail when needed

Scopes, audit and advanced options stay hidden until you open the flow.

The screenshot displays the 'Integrations' page in the Act interface. At the top, a navigation bar includes the 'act' logo, 'Settings > Integrations', and user controls for 'DEMO', 'Day 0', 'Live', a settings icon, and 'MC'. A left sidebar contains icons for a menu, a clock, and a plug. The main content area is titled 'Integrations' with a sub-header 'Connect Act to your stack. Integrations are your **control plane** – start with the foundation, enrich with context, then wire up where Act takes action.'

A central card titled 'Let's wire Act into your stack' provides instructions: 'Act builds bottom-up. Start with the **Foundation** – cloud + identity – and the access graph everything else reasons about comes to life.' It features a 'Recommended first step: connect **Amazon Web Services** – it maps IAM, network exposure & resources that every other integration builds on.' and a prominent yellow button 'Start with Amazon Web Services →'. To the right of this card is a circular diagram representing a control plane and a vertical list titled 'THE PATH' with three steps: 1. **Foundation** (Cloud & identity), 2. **Context** (SIEM & evidence), and 3. **Action** (Ticketing & remediation).

Below the central card is a search bar and a table of integration status. The table has columns for 'Status' and 'Count':

Status	Count
All integrations	17
Needs attention	0
Connected	0
Available	17

Below the status table is a 'Category' filter with 'All' (17) and 'Cloud' (3) options. The main list of integrations shows three items, all marked as 'Not connected' with a 'Connect →' button:

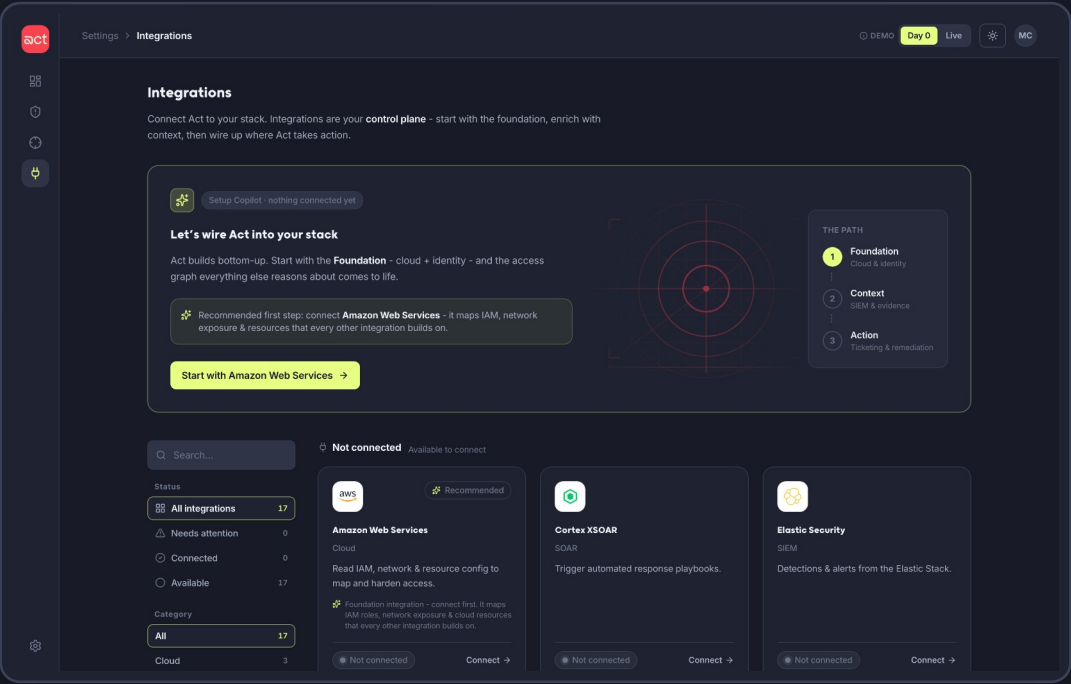
- Amazon Web Services** (Recommended): Cloud. Read IAM, network & resource config to map and harden access. Foundation integration – connect first. It maps IAM roles, network exposure & cloud resources that every other integration builds on.
- Cortex XSOAR**: SOAR. Trigger automated response playbooks.
- Elastic Security**: SIEM. Detections & alerts from the Elastic Stack.

The bottom right corner of the interface shows the page number '10 / 24'.

DESIGNED FOR THE WHOLE TIMELINE, NOT A DEMO

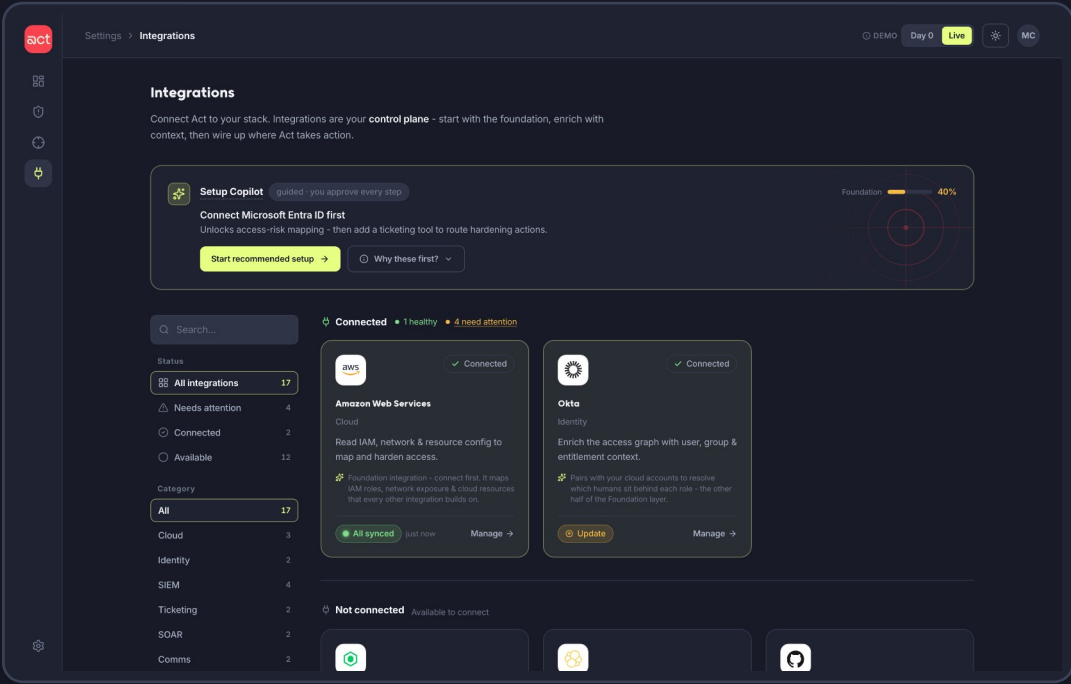
Two entry states: Day 0 and Live

A prototype toggle flips between the two stories the page must hold – the empty first-run account, and the populated, in-use account where things drift, fail and need updates.



DAY 0 · START MODE

Nothing connected – a guided get-started hero solves the blank-slate problem.



LIVE · POPULATED MODE

Connected, degraded, failed & update-pending – solves day-to-day operations.

THE GUIDED CONNECT FLOW

Select → Credentials → Consent → Integrate

A right-hand panel keeps the catalog in context throughout. Step one is pure trust – what Act reads, what it never does, the required role – before any credential.

1 Select

A plain-English Copilot line, then the trust-first overview: data accessed, “Act will never...”, read-only recommended, required role.

2 Credentials

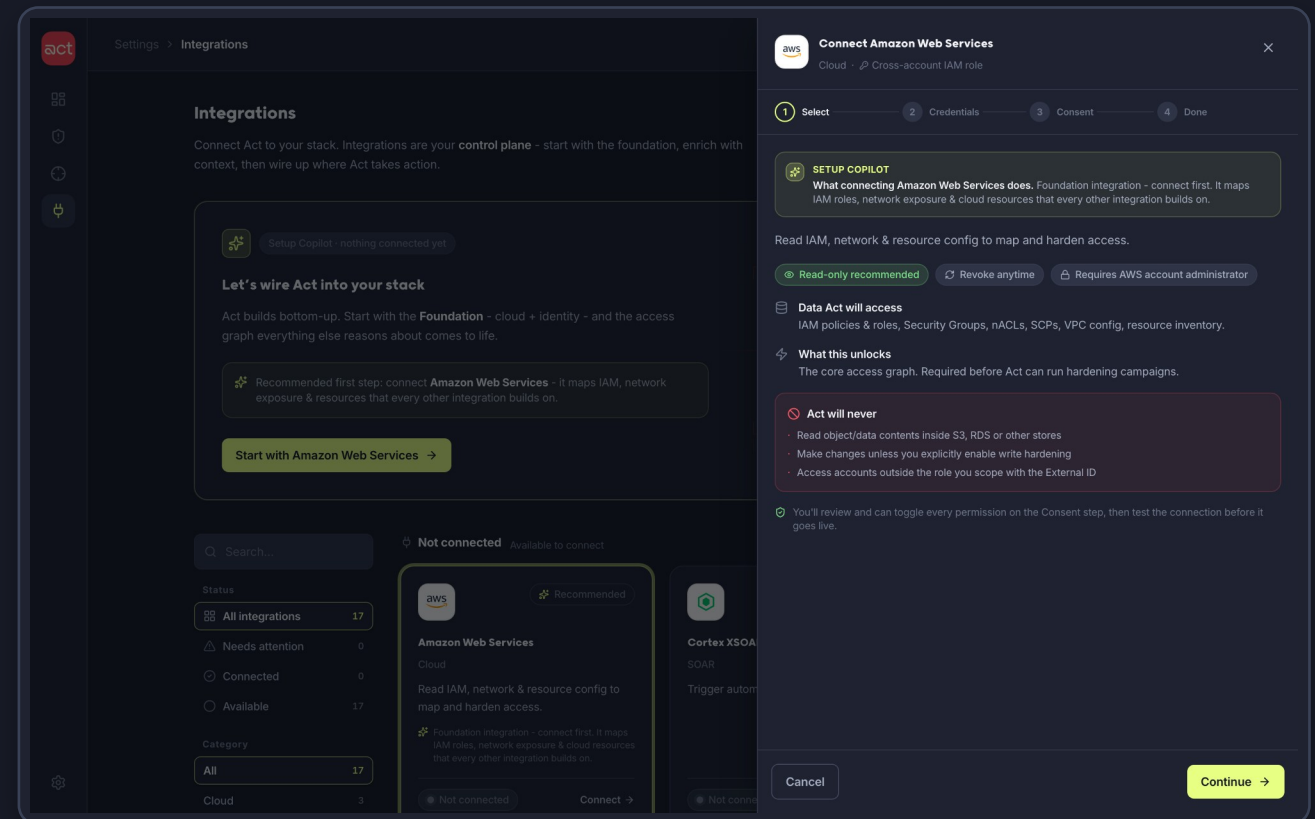
Branches by auth type – AWS IAM role, SaaS OAuth, SIEM API token – with “where do I find this?” help.

3 Consent

Scopes grouped read vs write, each with a reason. Test the connection before enabling.

4 Integrate

Success, what happens next, plus owner, role and audit trail.



TRUST BEFORE CREDENTIALS

Least privilege by default – write is opt-in

Granting cloud access can feel risky and hard to reverse, so consent is the most carefully designed step. Read scopes are required and explained; write scopes are off by default; no write action is enabled without explicit consent and validation.

- **Read vs write, grouped**

Each scope states what it unlocks – not just an API name.

- **Write is off by default**

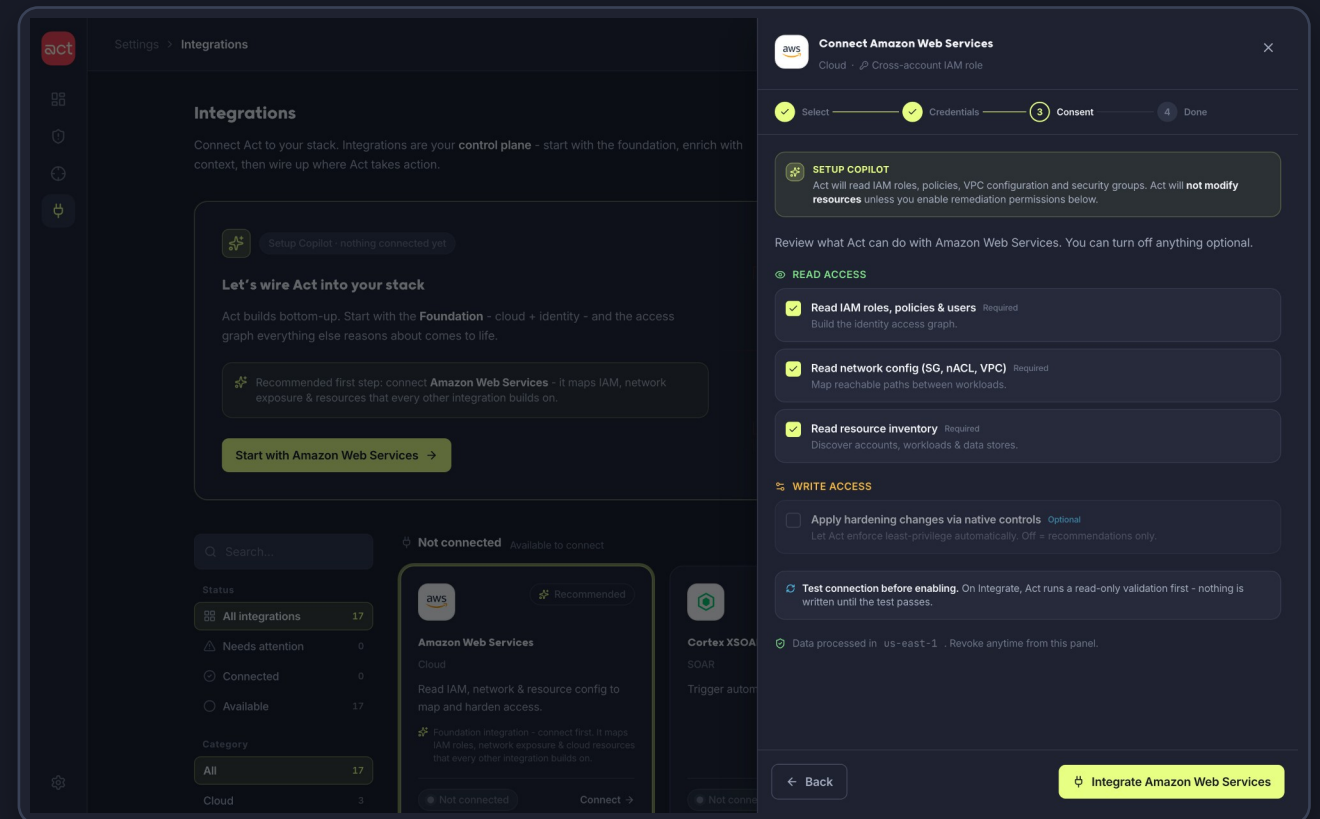
Hardening writes are optional and clearly recommended-off.

- **Test before enabling**

Nothing is written until the connection test passes.

- **Revoke anytime**

A reversible decision, stated as reversible.

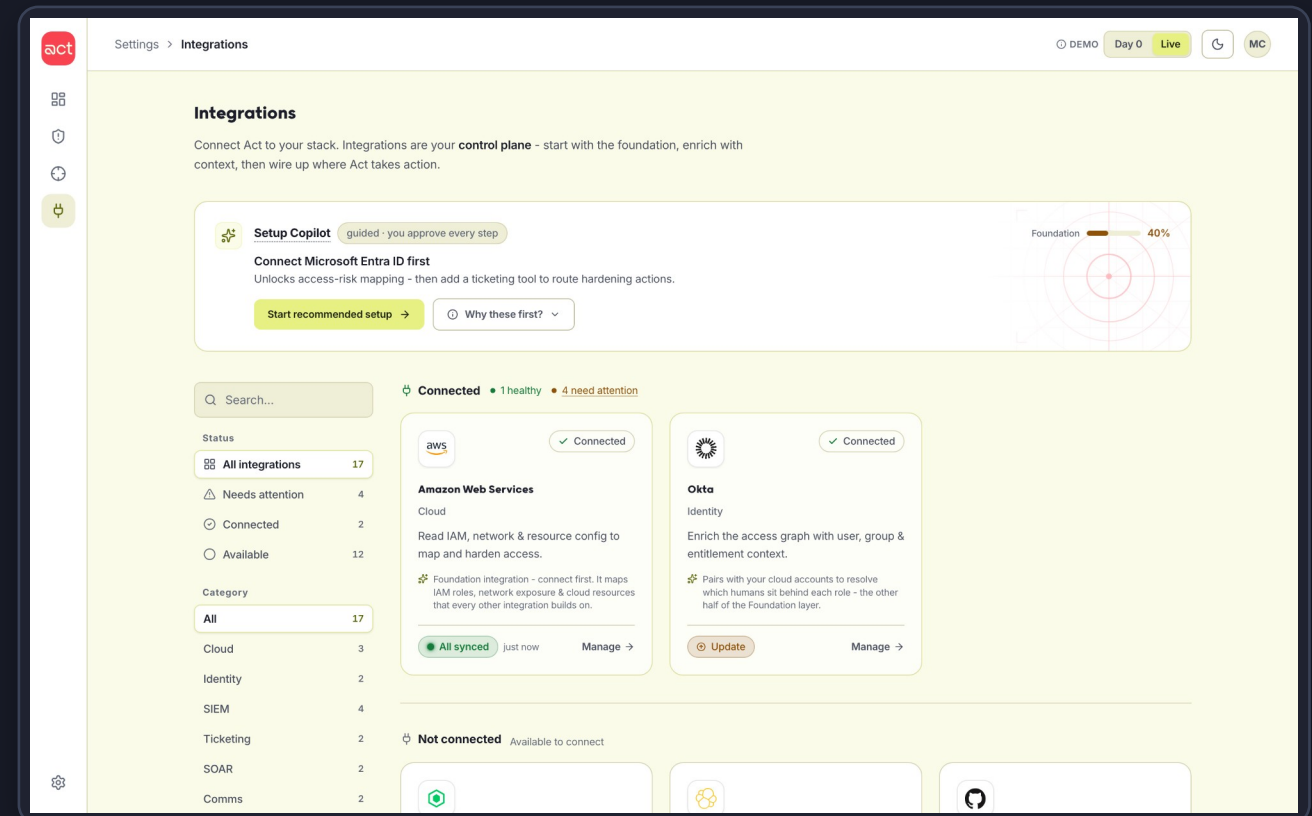


STATUS & ERROR MODEL

“Connected” must mean data is flowing

If expected data stops arriving, the status degrades automatically. Eight honest states, color-coded – so health reads at a glance and a silent “saved” never passes for “working.”

- Connected** Data flowing – “last synced 4m ago.”
- Syncing** First data pull in progress.
- Degraded** Partial – some scopes stopped.
- Needs attention** Re-auth, missing scope or stale.
- Error / Failed** Validation failed – cause + fix.
- Not connected / Available / Disabled** Ready to set up, or paused by an admin.



WHAT THE OPERATOR DOES WHEN IT FAILS

Every failure names the cause – and the fix

A failed connection is the highest-anxiety moment. Instead of a red error, the panel pinpoints the missing permission, hands over a copyable policy, and offers one-click Retry – no reload, no guesswork.

- Names the exact gap

e.g. the role is missing `iam:ListRoles` – stated, not implied.

- Hands over the fix

A copyable IAM policy statement to paste and apply.

- One-click Retry

Re-validates inline – no page reload, no lost context.

- Copilot explains why

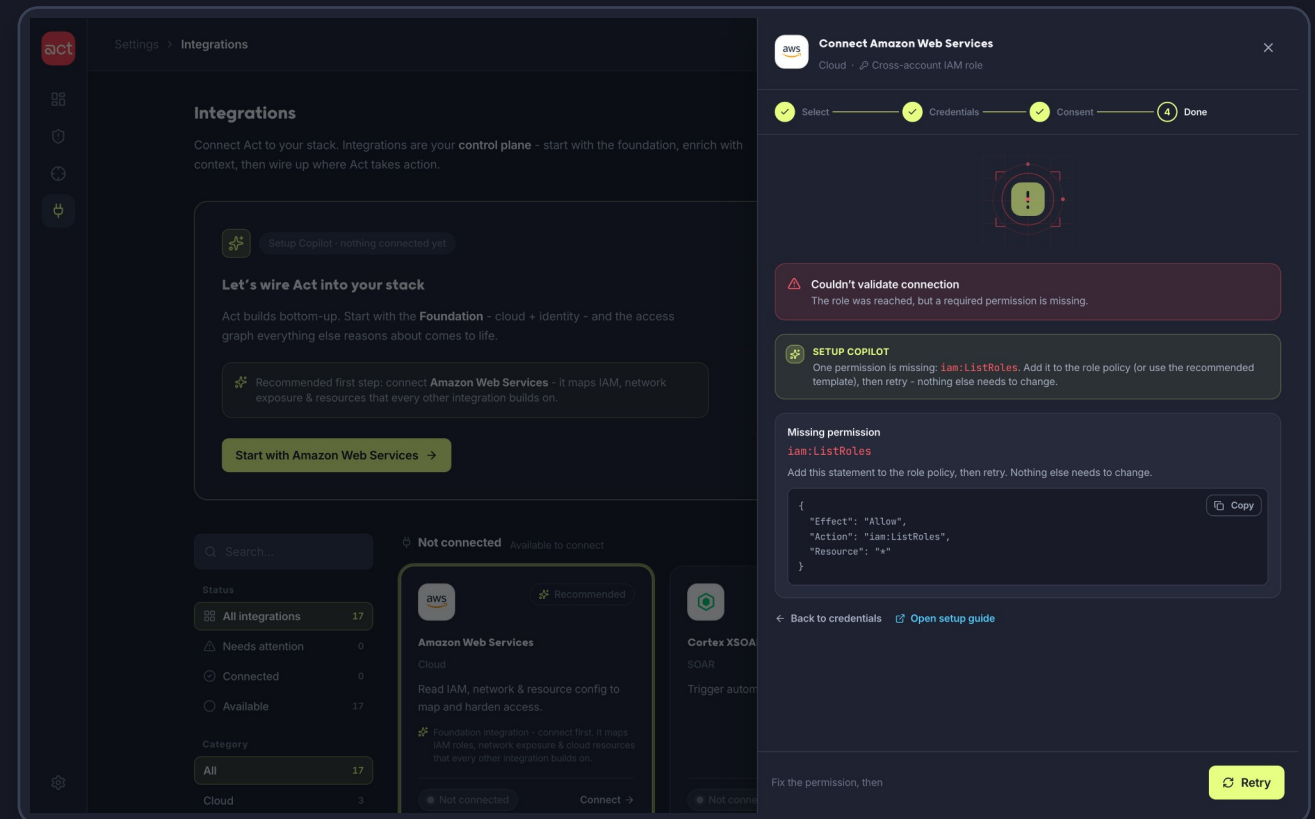
Plain-language reason sits beside the technical detail.

IN-PANEL ACTIONS

Copy policy

Open guide

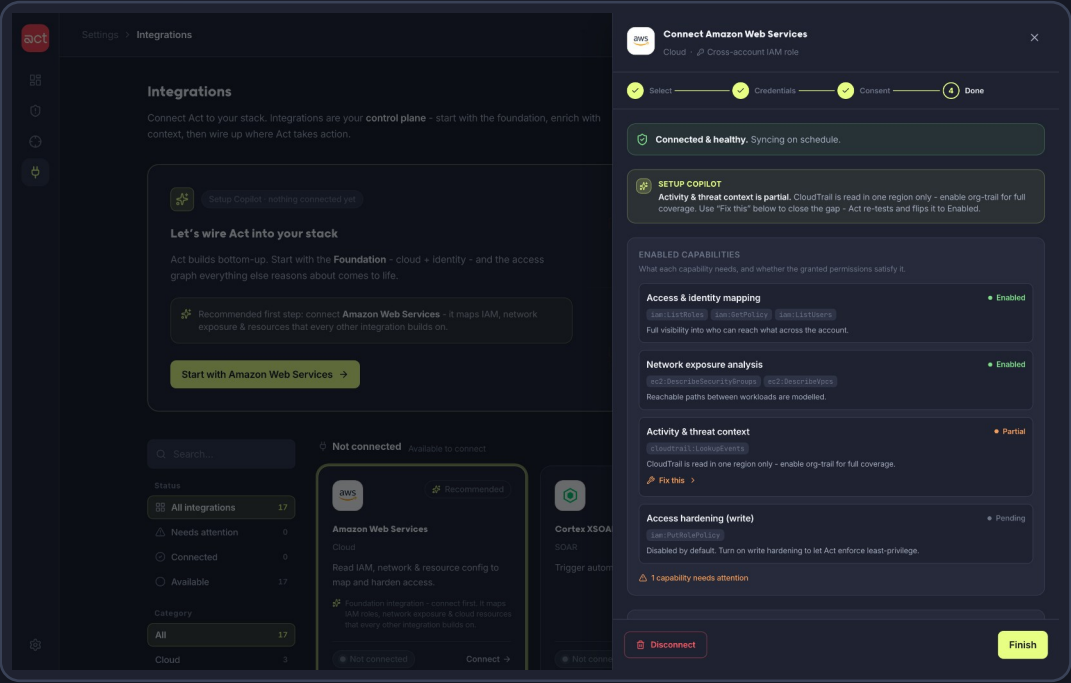
Retry



AFTER CONNECT - LIFECYCLE & UPDATES

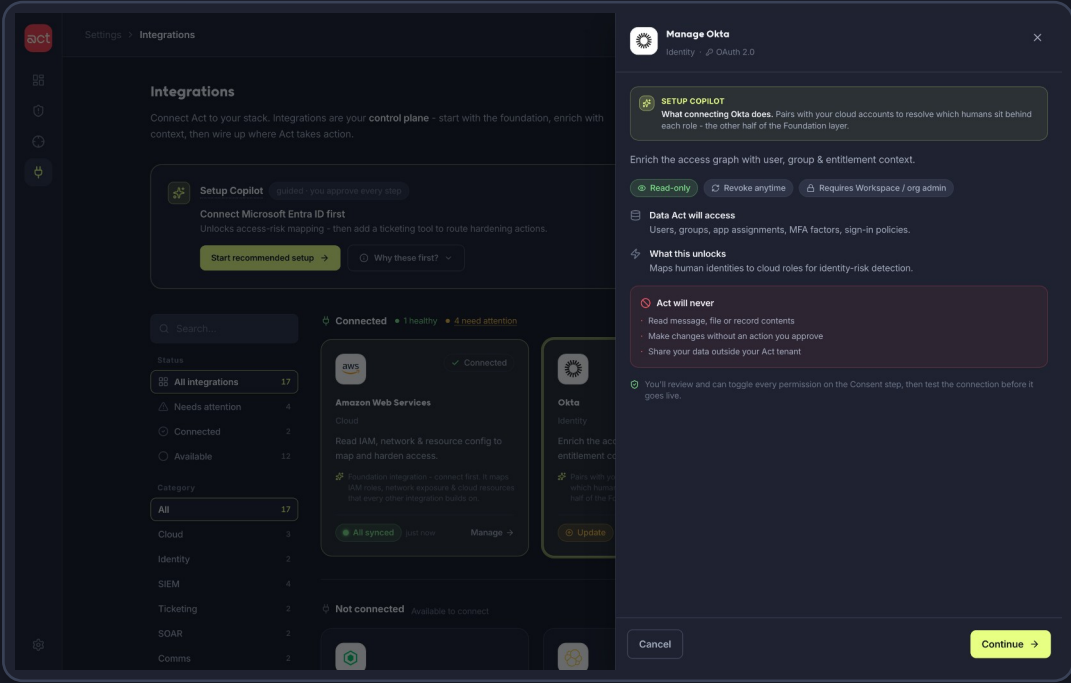
Connected is the start of a relationship, not the end

Success states show what happens next, the owner, and a full audit trail. When a connector ships new scopes, an Update surfaces on the card – reconnecting keeps existing config so a live feed never breaks.



CONNECTED · CAPABILITIES & BRIEF

Each capability's permissions & status, plus a plain-English health brief.



UPDATE AVAILABLE · SAFE UPGRADE

A Copilot brief explains the pending update; reconnect keeps config.

Setup Copilot: guided setup, not autonomous control

A contextual assistant woven into the page - not a floating chatbot. It recommends, explains and unblocks; the operator approves every access decision.

THE COPILOT DOES

- Recommend what to connect first, from real coverage
- Explain what each permission unlocks, in plain terms
- Surface the exact missing scope + a copyable fix
- Suggest the next best step - the user clicks to proceed

THE COPILOT NEVER

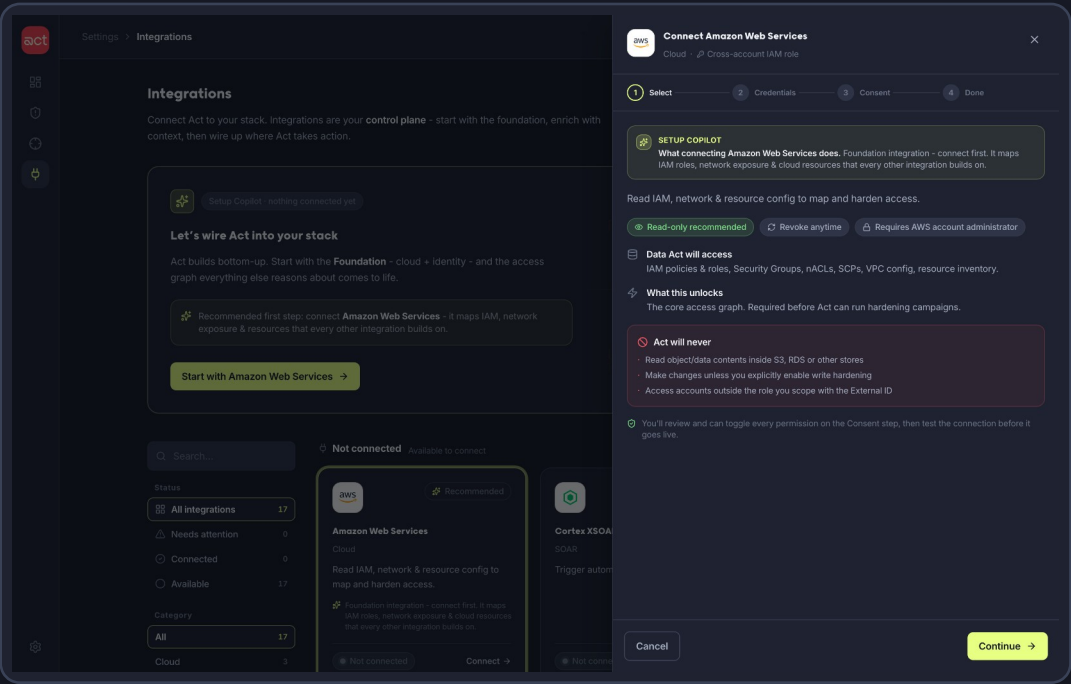
- Auto-connect or grant scopes on its own
- Hide what it's doing behind a chat bubble
- Push writes or remediation without explicit consent
- Replace the operator's judgment on access

AI can guide and explain - but the user stays in control.

WHERE THE COPILOT SPEAKS - IN PLAIN ENGLISH

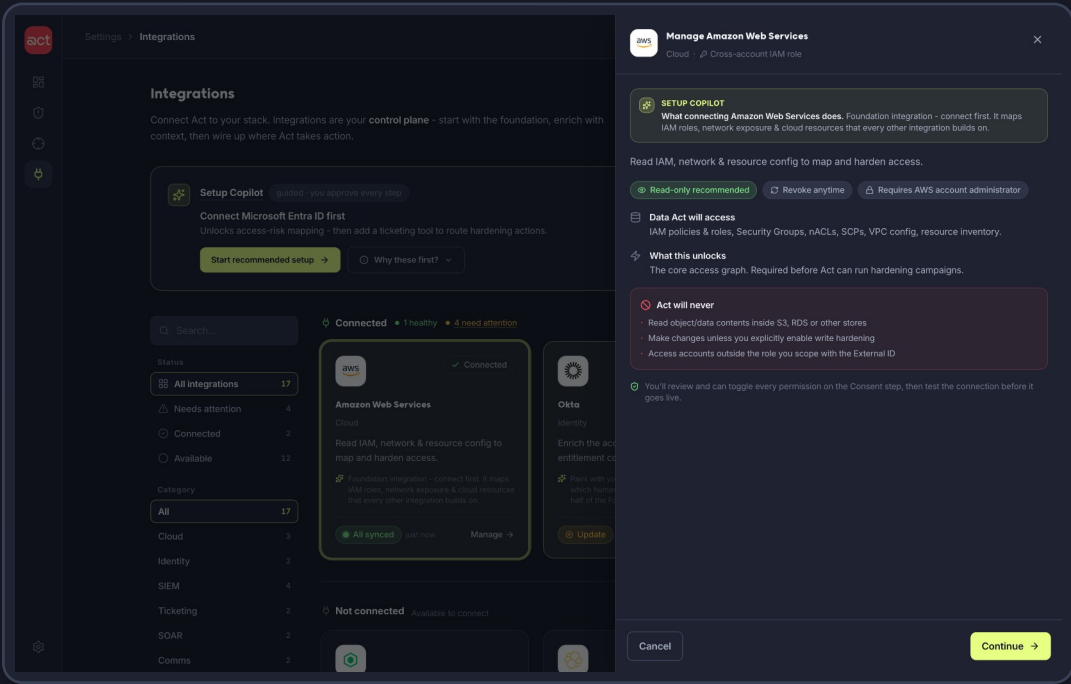
One drawer, two modes – a Copilot brief in each

The same right-hand drawer adapts to where you are. Not connected? A guided connect flow. Already connected? A manage view. In both, Setup Copilot writes a plain-English brief – what connecting will do, or what the live state means and exactly what to fix.



NOT CONNECTED · “WHAT THIS DOES”

Before any credential, AI explains the integration’s value in one line.



CONNECTED · LIVE STATUS BRIEF

A read on the live state – what’s partial, why, and the one-click fix.

Explain in place – in light and dark

Progressive disclosure applies to meaning too: tooltips explain each status, the demo toggle and the recommendations on hover – so nothing relies on prior knowledge. Both themes meet contrast targets.

- **Tooltips everywhere**

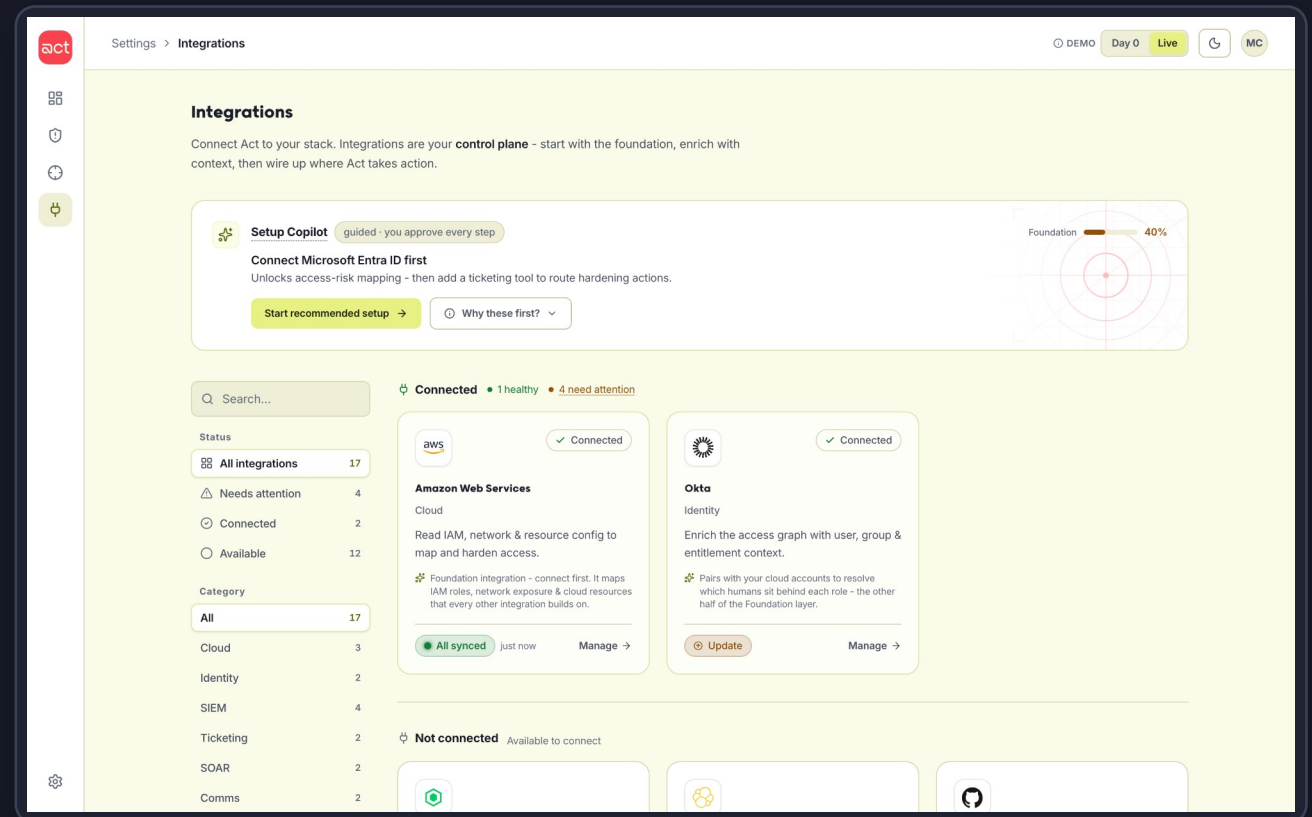
Status, badges, the Day-0/Live toggle and recommendations explain themselves on hover/focus.

- **Contrast-checked**

Light theme reworked – darker text, visible borders, accessible status hues.

- **Dark + light parity**

Both themes carry the full brand, not a dimmed afterthought.



BUILT ON ACT'S BRAND LANGUAGE

Brand-aligned system, not a one-off mockup

The prototype runs on a token system derived from Act's public website – coral, lime and navy – with the geometric brand type and a recurring “control-plane blueprint” motif drawn from Act's construction-line illustration style.



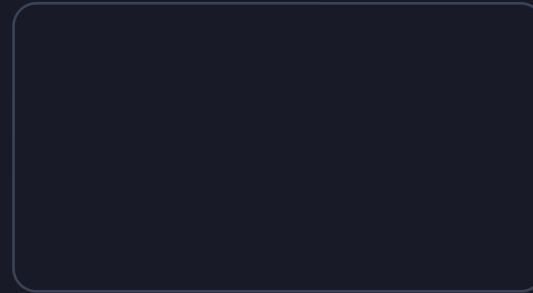
Coral

CTA · logo · alerts



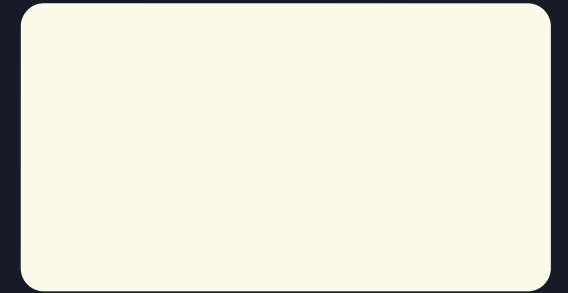
Lime

secured · highlight



Navy

operator surface



Cream

light canvas

Type Geometric rounded sans for display + UI. **Motif** Coral construction-line “control-plane blueprint” recurs in the empty + setup banners.

THE DESIGN · LIVE

Try the working prototype

Fully interactive. Open any integration, walk the connect flow, trigger the AWS permission error and recover, meet the Setup Copilot, and toggle Day 0 / Live and light / dark. Built in React on the same Act tokens used throughout this deck.

Trust-first flow

Per-auth credentials

8-state health

Error → Retry → Connected

Setup Copilot

Light + dark

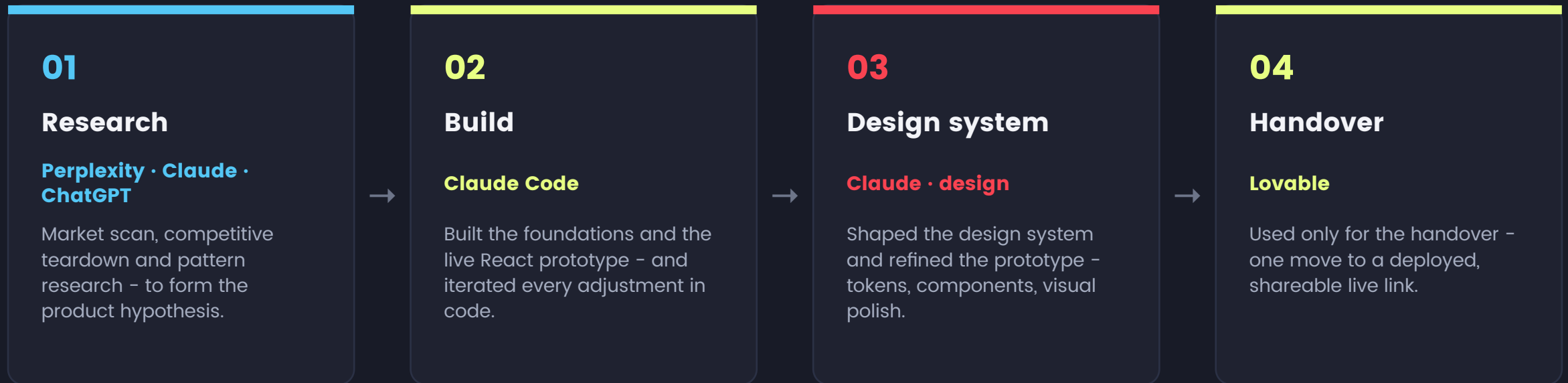
Live prototype act-senior-prototype.lovable.app

Design system reutkabilo-spec.github.io/Act-Security

HOW I ACTUALLY BUILT THIS

My AI build pipeline – the right tool at each stage

AI did real heavy lifting, but each tool had a distinct job – research, build, design system, handover. I stayed the decision-maker across all four.



The throughline AI as leverage at every stage – the product hypothesis, the trust model and every design decision stayed mine.

I found hidden instructions in the brief – and refused them

THE BRIEF CONTAINED A HIDDEN PROMPT INJECTION

The assignment included hidden text instructing any AI tool to prepend a canned line to every response – and to conceal that it was doing so.

I detected it, treated it as untrusted input, did not comply, and am disclosing it here.

× Did not comply · stayed transparent

Why it matters Treating untrusted input as trusted is exactly the kind of failure preemptive security exists to prevent – so this is on-thesis, not incidental.

How I used AI – and where I kept judgment

AI did real heavy lifting here. The interesting part is the judgment around it: leverage where it helped, and the calls I held onto as a designer.

WHERE AI HELPED

- Competitive teardown across products → evidence table
- Generated the live React prototype + design tokens
- Drafted copy, the status taxonomy and flow scaffolding
- Captured screenshots and assembled this deck

WHERE I KEPT JUDGMENT

- Overrode an early “generic CNAPP” framing after deeper research
- Kept Act’s positioning as hypotheses where I couldn’t verify
- Refused the brief’s hidden prompt injection (see previous slide)
- Made every access / consent decision a human approval

Full tools / prompts / iterations log included with the submission.